

# INFORME ANALITICO TECNICO SEMÁFOROS GRUPO N°1



## **Integrantes:**

Lezcano, Axel Antonio

Cardozo, Fabricio Hernan

Almirón, Franco Gonzalo

## Fundamentos sobre el diseño funcional adoptado:

### 1) Transición (Implementación final):

```
(defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal color-actual 'en-verde) (equal cambiar-a 'cambiar-a-amarillo)) (list color-actual 'verde-intermitente "CAMBIAR-A-AMARILLO" ))
    ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'cambiar-a-rojo)) (list color-actual 'amarillo-intermitente "CAMBIAR-A-ROJO"))
    ((and (equal color-actual 'en-rojo) (equal cambiar-a 'cambiar-a-verde)) (list color-actual 'rojo-intermitente "CAMBIAR-A-VERDE" ))
    (t (list color-actual 'accion-por-defecto ))
  )
)
```

La lógica utilizada en el siguiente código y la forma en la cual fue implementada de la siguiente forma: para la modelación de los distintos cambios del semáforo (teniendo en cuenta las entradas por parámetro asignadas al comienzo “color-actual” y “cambiar-a”) se utilizó una estructura de alternativa múltiple (cond), donde cada caso representaba un caso posible y válido del ciclo (rojo→verde→amarillo), el cual mediante un operador lógico *and* se fue verificando que cumpliera justamente ese mismo cambio. Luego con la llegada de la intermitencia por cambios de colores se actualizó la implementación de la función, colocando únicamente la intermitencia correspondiente en cada caso. Ahora, ¿por qué decidimos desarrollarlo de esta forma? la función transicion justamente es una función la cual es invocada únicamente por el cambio de color que ocurre en el ciclo, ejemplo: cambiar de **verde**→**amarillo**, entonces la transición en esta parte debería de indicar esa simulación de intermitencia entre el cambio de verde a amarillo, simulando lo siguiente: **verde**→**verde-intermitente**→**amarillo**, dando así como concluida esta función.

### 2) Timer (implementación final)

```
(defun timer (timestamp) ;cuando tienes un semáforo, al terminar el tiempo de rojo, no se le suma +3 de intermitencia al rojo
                          ;la intermitencia empieza a cambiar en los últimos 3 segundos de cada color
  (cond
    ((<= 0 (mod timestamp 216) 86) 'en-rojo )
    ((<= 87 (mod timestamp 216) 89) 'en-rojo-intermitente )
    ((<= 90 (mod timestamp 216) 206) 'en-verde )
    ((<= 207 (mod timestamp 216) 209) 'en-verde-intermitente )
    ((<= 210 (mod timestamp 216) 212) 'en-amarillo )
    (t 'en-amarillo-intermitente)
  )
)
```

La lógica implementada en este requerimiento fue de la siguiente manera: al principio con únicamente 4 tipos de casos distintos respecto a los colores (rojo, verde y amarillo), se calculaba el resto del tiempo UNIX, el cual accedía a la función mediante el parámetro, para calcular el tiempo exacto en donde estábamos parados en ese último ciclo, comenzando desde el 0 hasta 215 que eran todos los casos posibles, siendo el 0 el segundo 1 del ciclo y 215 el segundo 216. De esta forma los intervalos de números entre ellos representaban el segundo (n+1) del ciclo. Luego con la nueva implementación con la llegada de la intermitencia llegamos a una conclusión, la intermitencia ocurre en los últimos segundos de cada color, ya que al terminar un color del semáforo no se le incluye +3 segundos.

adicionales al color para cambiar, si no que al ir terminando un color entra al estado de intermitencia al entrar en esos 3 segundos siendo así y dando como resultado la función introducida más arriba, con sus propios casos a los colores de intermitencia.

### 3) *loginLights (implementación final):*

```
(defun loginLights (color-actual cambio-color)
  (format t "Tiempo ~A: la luz ah cambiado de color ~A a ~A"
    (local-time:format-timestring nil (local-time:now):format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2)))
    (car (transicion color-actual cambio-color)) (caddr (transicion color-actual cambio-color))
  ) )
```

Para el procedimiento de esta función de auditoría, al principio imprimimos en pantalla el tiempo (utilizando get-universal) y el cambio del color-actual al cambiar-color. Luego actualizamos la función de local-time, tuvimos una mejor representación sobre el tiempo, siendo más legible para el lenguaje humano y utilizando la función transición como una forma de asegurar de que el cambio el cual se va a realizar es el correcto.

### 4.a) *duración-ciclo (implementación final):*

```
(defun duracion-ciclo (rojo verde amarillo)
  (+ rojo verde amarillo)
)
```

En esta parte, según lo comprendido por el grupo, se entendió que la duración del ciclo total va a variar dependiendo de los colores con los que se quiera trabajar en el semáforo. Es una función la cual ayuda a saber cuánto va a durar un ciclo según esos colores.

### 4.b) *Recomendacion-ciclo (implementacion final):*

```
(defun recomendacion-ciclo (duracion)
  (if (and (>= duracion 35) (<= duracion 150))
    "ciclo optimo"
    "ciclo no optimo"
  )
)
```

Esta función se encarga a través de una condicional simple y un operador lógico *and* si la duración del ciclo se encuentra entre los rangos 35 hasta 150. Teniendo esta función como dato de entrada en su parámetro a la función *duracion-ciclo* la cual se habló anteriormente y se encargará de sumar y calcular si está o no en el rango óptimo.

### 5) *ciclos-por-tiempo(implementación final):*

```
(defun ciclos-por-tiempo(minutos)
  (print "La cantidad de ciclos es de:")
  (print (truncate (/ (* minutos 60) 216))); truncate toma el resultado de una operacion y elimina el decimal, si el resultado es 28.9, quedaria 28
)
```

En esta función se imprime por pantalla la cantidad de ciclos completos los cuales pueden ser almacenados a la cantidad de minutos la cual es llegada como parámetro, la estrategia fue simple y sencilla: multiplicar la cantidad de minutos por 60 para transformarlos a la unidad de segundos, luego dividimos ese tiempo por 216 (cantidad de segundos que ocupa un ciclo) y lo truncamos, esto debido que queremos la cantidad de ciclos enteros.

## 6) *calcularRestoIni (Función Auxiliar):*

```
(defun calcularRestoIni (restoIni)
  (cond
    ((<= 0 restoIni 86) (list (- 86 restoIni) 3 117 3 3 3)) ;(- 86 restoIni) --> indica lo consumido por rojo
    ((<= 87 restoIni 90) (list 0 (- 90 restoIni) 117 3 3 3)) ;(- 90 restoIni) --> indica lo consumido por el rojo-intermitente
    ((<= 91 restoIni 206) (list 0 0 (- 206 restoIni) 3 3 3)) ;(- 206 restoIni) --> indica lo consumido por verde
    ((<= 207 restoIni 209) (list 0 0 0 (- 209 restoIni) 3 3)) ;(- 209 restoIni) --> indica lo consumido por verde-intermitente
    ((<= 209 restoIni 212) (list 0 0 0 0 (- 212 restoIni) 3)) ;(- 212 restoIni) --> indica lo consumido por amarillo
    ((<= 212 restoIni 215) (list 0 0 0 0 0 (- 215 restoIni))) ;(-215 restoIni) --> indica lo consumido por amarillo-intermitente
  )
)
```

En esta se basó en los cálculos obtenidos de anteriormente antes de ser introducidas las intermitencias. En resumen sobre los cálculos de esta función, se devuelve una lista para poder retornar las cantidades exactas de los segundos consumidos según cada caso. Al ser una función la cual le llegaba como parámetros el resto inicial del tiempo unix (explicada más) nos daba el segundo actual en donde estábamos parados en ese intervalo de tiempo (pudiendo ser del 0-215), tomando ese tiempo se calculaba cuánto tiempo consumido hay desde ese instante hasta el fin de ese ciclo (para el cálculo de porcentajes más adelante).

### 6.1) *calcularRestoFin (Función Auxiliar):*

```
(defun calcularRestoFin (restoFin)
  (cond
    ;; Al final de la hora es al revés: calculamos cuánto se consumió desde el inicio de ese último ciclo incompleto hasta llegar al segundo 'restoFin'
    ((<= 0 restoFin 86) (list restoFin 0 0 0 0 0)) ;(- 86 restoFin) --> indica lo consumido por rojo
    ((<= 87 restoFin 89) (list 87 (- restoFin 87) 0 0 0 0)) ;(- 87 restoFin) --> indica lo consumido por el rojo-intermitente
    ((<= 90 restoFin 206) (list 87 3 (- restoFin 90) 0 0 0)) ;(- 90 restoFin) --> indica lo consumido por verde
    ((<= 207 restoFin 209) (list 87 3 117 (- restoFin 207) 0 0)) ;(- 207 restoFin) --> indica lo consumido por verde-intermitente
    ((<= 210 restoFin 212) (list 87 3 117 3 (- restoFin 210) 0)) ;(- 210 restoFin) --> indica lo consumido por amarillo
    ((<= 213 restoFin 215) (list 87 3 117 3 3 (- restoFin 213))) ;(- 213 restoFin) --> indica lo consumido por amarillo-intermitente
  )
)
```

Similar a la función anterior, pero en este caso el parámetro resto Fin es el resto de la hora unix + 3600 (la siguiente hora). la única diferencia entre esta y la anterior, es que *calcularRestoFin* devuelve una lista donde los consumidores ya no están adelante, si no desde el resto parado hacia el comienzo de ese ciclo (ya no hacia el final).

## 6.2) calcularPorcentajes (Función Auxiliar):

```
(defun calcularPorcentajes (ListaIni ListaFin)
  (list
    (float (/ (* (+ 1392 (nth 0 ListaFin) (nth 0 ListaIni)) 100) 3600)) ;rojo
    (float (/ (* (+ 48 (nth 1 ListaFin) (nth 1 ListaIni)) 100) 3600)) ;rojo-intermitente
    (float (/ (* (+ 1872 (nth 2 ListaFin) (nth 2 ListaIni)) 100) 3600)) ;verde
    (float (/ (* (+ 48 (nth 3 ListaFin) (nth 3 ListaIni)) 100) 3600)) ;verde-intermitente
    (float (/ (* (+ 48 (nth 4 ListaFin) (nth 4 ListaIni)) 100) 3600)) ;amarillo
    (float (/ (* (+ 48 (nth 5 ListaFin) (nth 5 ListaIni)) 100) 3600)) ;amarillo-intermitente
  )
)
```

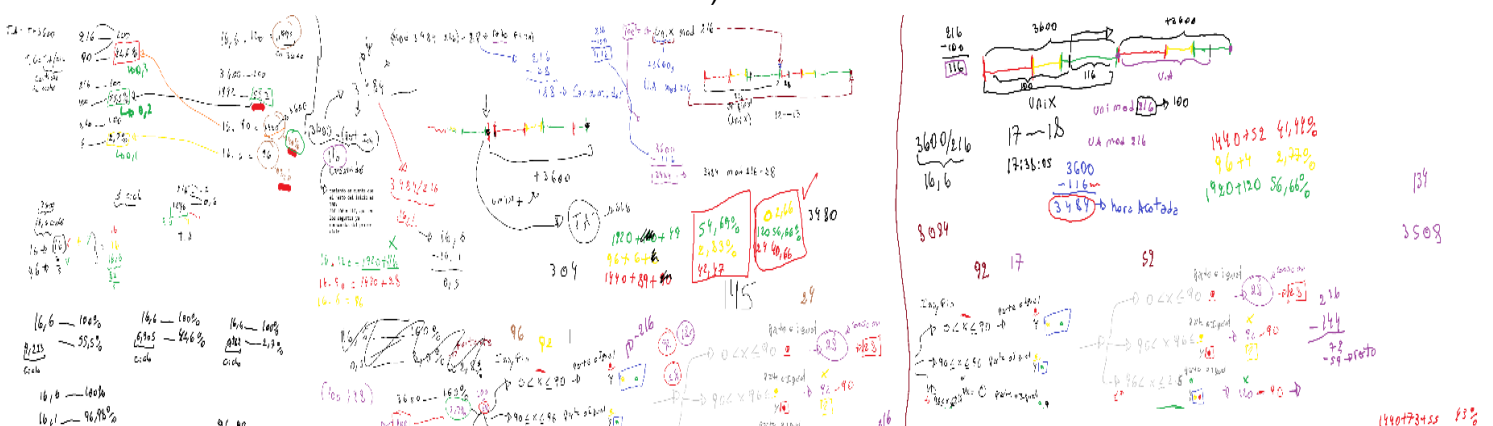
Se aplica el cálculo de la regla de 3 simples, accediendo a cada elemento con el nth (los elementos se muestran a la derecha en forma de comentarios) y pasandolos a float en el debido caso que los resultados dieran con coma. Además para la suma de porcentajes, se le quita la equivalencia de los intermitentes según sus respectivos colores (ejemplo: el rojo en rangos de 1 hora equivale a 1440s, pero al quitar su parte intermitente, la cual es 48s, da como resultado el número 1392).

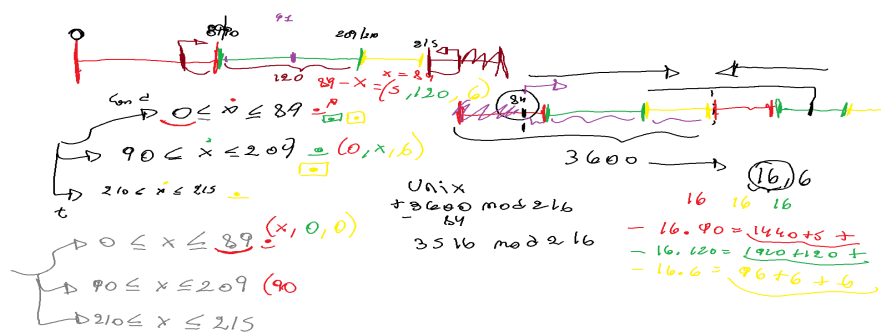
Por otra parte, se puede observar que los datos del parámetro son 2 listas, las cuales corresponden al retorno de las 2 últimas funciones auxiliares que vimos (calcularRestoIni y CalcularRestoFin).

## 6.3) distribucionTemp (Implementación Final):

```
(defun distribucionTemp (unix)
  (if (zerop (mod unix 216)) "| 40,28% rojo| 1,39% rojo-intermitente | 54,17% verde| 1,39% verde-intermitente| 1,39% amarillo| 1,39% amarillo intermitente|"
      (format t "~%| ~A% rojo| ~A% rojo-intermitente| ~A% verde | ~A% verde-intermitente| ~A% amarillo| ~A% amarillo-intermitente|"
        (nth 0 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
        (nth 1 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
        (nth 2 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
        (nth 3 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
        (nth 4 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
        (nth 5 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
      )
  )
)
```

Esta función no hace más que imprimir los porcentajes retornados por la función *calcularPorcentajes*, los cuales a su vez reciben a *CalcularRestoIni* y *CalcularRestoFin*. A continuación se mostrará los cálculos y pensamientos (pero más utilizado como C.A., o también conocidos como cálculos auxiliares).





## 7) Informe (Implementación Final):

```
(defun informe (color-actual cambio-color)
  (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output)
    (format stream "Informe de Ejecución del Sistema Semafórico-%")
    (format stream "-----~%")
    (loginLights color-actual cambio-color)
    (format stream "~A: transición: ~A --> ~A"
      (local-time:format-timestring nil (local-time:now):format '(:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2)))
    (car (transicion color-actual cambio-color)) (caddr (transicion color-actual cambio-color)) )
    (format stream "~% --- Fin del Informe ---") ) )
;pasandole login para ver en pantalla y luego la impresion dentro del archivo
```

La función informe se encarga de escribir en un archivo txt el cambio de color que ocurre en un determinado tiempo, el cual está expresado de una forma fácilmente entendible para el humano.

En primer lugar la función abre el archivo, y luego con los format stream que aparecen le dan la instrucción al sistema que escriba dentro del archivo txt los distintos string que aparecen adjuntos al format, en por medio se llama a la función loginLights para que además de guardar los datos en el txt, también lo imprima en pantalla para que el usuario pueda ver que es lo que se guardó. Por consiguiente volvemos a toparnos con un format que se encarga de imprimir en el archivo la hora con el **local-time**, se le indica que queremos que el tiempo sea impreso en formato string le damos el tiempo en "crudo" el tiempo, luego de esto le indicamos cuantos caracteres queremos que se escriba de cada tipo (ejemplo, para los años le indicamos que queremos 4 "2026" por ejemplo, los meses 2 "06" días "14" hora "17" minutos "30". Luego de todo eso se indica el primer elemento de la lista de transición como una forma de asegurar que el valor es el indicado y el último elemento.

## Respuestas sobre de las preguntas sobre Scala:

- 1. Scala combina la Programación Orientada a Objetos (POO) con la Funcional. ¿Cómo estructuraron el semáforo? ¿Usaron una clase tradicional, un **object** (Singleton), o **case classes**? Justifiquen desde el diseño funcional.

- 2. Comparen la manipulación de listas en Scala (métodos como `.map` o `.filter`) contra las funciones de orden superior de Common Lisp. ¿Cuál resulta más legible y por qué?
- 
1. Con respecto a la estructura del semáforo, no utilizamos clases como el objet ni case clases. El programa se diseñó con funciones puras, en donde cada función responde según sus parámetros de entrada, resolviendo los problemas de manera funcional, devolviendo resultados sin modificar ningún tipo estructuras externas e internas
  2. Investigando sobre el tema, los métodos mencionados de `.map` y `.filter`, es mucho más fácil y resumido de entender que la sintaxis del common lisp. ejemplo:  
lisp: (remove-if-not #'(lambda (x) (> x 0)) lista)  
scala: lista.filter(x => x > 0)  
para alguien que mira estas sintaxis por primera vez, es mucho más fácil descifrar y de entender la sintaxis de scala, siendo más directo respecto a los nombres y funciones. Por otra parte, la ventaja que presentó Scala respecto a lisp para el grupo, fue el gran parecido que tiene con el lenguaje de C + + en su sintaxis y no utilizando un lenguaje polaco como lo es lisp.

Fase 2:

Se decidió utilizar la librería (local-time) del gestor de paquetes de quicklisp, debido a la facilidad que esta nos resultó para implementarla en las funciones informe y loginLights a comparación de la biblioteca (cl-json), que para los integrantes, resultó algo tedioso y confuso, combinar el archivo core.lisp junto con el archivo config.json

**Bitácora de debug de depuración:**

**Commit f9660ce:**



```

313 +
314 + (defun logginlights (color-actual cambio-color)
315 +   ((format nil "Tiempo ~0: la luz ah cambiado de color ~5 a ~5" (- (get-universal-time) 2208988800) color-actual cambio-color)
316 + )
318 317 ;Extension 2, sistema de datos
318 + (informe (list (logginlights 'en-rojo 'en-rojo-intermitente) (logginlights 'en-rojo-intermitente 'en-verde)
319 + (logginlights 'en-verde 'en-verde-intermitente) (logginlights 'en-verde-intermitente 'en-amarillo)
320 + (logginlights 'en-amarillo 'en-amarillo-intermitente) (logginlights 'en-amarillo-intermitente 'en-rojo)
321 + )
322 + )
323 +
319 324 (defun informe (datos)
320 - (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output)
321 - (format stream "Informe de Ejecución del Sistema Semafórico~%")
322 - (format stream "=====~%")
323 - (mapcar #'(lambda (x) (format "~A~%" stream) x) datos)
324 - (format stream "~%--- Fin del Informe ---"))
325 + (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output)
326 + (format stream "Informe de Ejecución del Sistema Semafórico~%")
327 + (format stream "=====~%")
328 +
329 + (mapcar #'(lambda (x) (format stream "~A~%" x) datos)
330 + ;; Implementar iteración sobre datos y formateo
331 + ;; Ejemplo de línea: "2024-06-04 14:38:15 - Transición: ROJO → VERDE"
332 + (format stream "~%--- Fin del Informe ---")
333 + )
334 + )

```

En este commit, se quiso realizar la función para iteración 2 de la extensión 2, en el que se puede ver que se abría un archivo y lo imprimía en el archivo.txt, pero este se hacía a través de la función mapcar (que es una función de orden superior) , en lo que generaba un error ya que imprimía todos los casos posibles la misma hora en el archivo.txt.

## Commit 0d6b4bc:

```

294 294 )
295 295 ;-----
296 296 #|(defun distribucionTemp (unix)
297 297 (if (zerop (mod unix 216)) "[ 40,28% rojo | 1,39% rojo-intermitente | 54,17% verde | 1,39% verde-intermitente | 1,39% amarillo | 1,39% amarillo intermitente]"
298 298 (format t "~%| ~A rojo | ~A rojo-intermitente| ~A verde | ~A verde-intermitente| ~A amarillo| ~A amarillo-intermitente|"
299 299 (car (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
300 300 (cadr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
301 301 (caddr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
302 302 (caddr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
303 303 (caddr (cdr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
304 304 (car (last (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
305 305 )
306 306 )|#
307 307
308 308 (print (distribucionTemp 3600))
309 309
310 + ;actualizado
311 + (defun distribucionTemp (unix)
312 + (if (zerop (mod unix 216)) "[ 40,28% rojo | 1,39% rojo-intermitente | 54,17% verde | 1,39% verde-intermitente | 1,39% amarillo | 1,39% amarillo intermitente]"
313 + (format t "~%| ~A rojo | ~A rojo-intermitente| ~A verde | ~A verde-intermitente| ~A amarillo| ~A amarillo-intermitente|"
314 + (nth 0 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
315 + (nth 1 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
316 + (nth 2 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
317 + (nth 3 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
318 + (nth 4 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
319 + (nth 5 (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
320 + )
321 + )
310 322
311 323 ;Extension 2, sistema de datos
312 324

```

En este commit, se corrige la función (distribuciontemp) y se hace cambios reemplazando las funciones predefinidas car y cdr, por las de NTH, permite extraer ese elemento de esa



posición específica en la memoria, a diferencia de car y cdr, que podría generar ciertos errores basura (como porcentajes incorrectos de los colores) o duplicar valores.

## Commit 42b4a0a

```
193 193 IMPACTO: No destructiva
194 194 -----|#
195 195
196 196 #|(defun calcularRestoFin (restoFin)
197 197   (cond
198 -   ((<= 0 restoFin 89) (list (- 89 restoFin) 3 117 3 3 3)) ;(- 86 restoFin --> indica lo consumido por rojo
199 -   ((<= 87 restoFin 90) (list 0 (- 90 restoFin) 117 3 3 3)) ;(- 90 restoFin --> indica lo consumido por el rojo-intermitente
200 -   ((<= 91 restoFin 206) (list 0 0 (- 206 restoFin) 3 3 3 3)) ;(- 206 restoFin --> indica lo consumido por verde
201 -   ((<= 207 restoFin 209) (list 0 0 0 (- 209 restoFin) 3 3 3)) ;(- 209 restoFin --> indica lo consumido por verde-intermitente
202 -   ((<= 209 restoFin 212) (list 0 0 0 0 (- 212 restoFin) 3 3 3)) ;(- 212 restoFin --> indica lo consumido por amarillo
203 -   ((<= 212 restoFin 215) (list 0 0 0 0 0 (- 215 restoFin))) ;(- 215 restoFin --> indica lo consumido por amarillo-intermitente
198 +   ;; Al final de la hora es al revés: calculamos cuánto se consumió desde el inicio de ese último ciclo incompleto hasta llegar al segundo 'restoFin'
199 +   ((<= 0 restoFin 86) (list restoFin 0 0 0 0 0)) ;(- 86 restoFin --> indica lo consumido por rojo
200 +   ((<= 87 restoFin 89) (list 87 (- restoFin 87) 0 0 0 0 0)) ;(- 87 restoFin --> indica lo consumido por el rojo-intermitente
201 +   ((<= 90 restoFin 206) (list 87 3 (- restoFin 90) 0 0 0 0 0)) ;(- 90 restoFin --> indica lo consumido por verde
202 +   ((<= 207 restoFin 209) (list 87 3 117 (- restoFin 207) 0 0 0 0 0)) ;(- 207 restoFin --> indica lo consumido por verde-intermitente
203 +   ((<= 210 restoFin 212) (list 87 3 117 3 (- restoFin 210) 0 0 0 0 0)) ;(- 210 restoFin --> indica lo consumido por amarillo
204 +   ((<= 213 restoFin 215) (list 87 3 117 3 3 (- restoFin 213))) ;(- 213 restoFin --> indica lo consumido por amarillo-intermitente
204 205   )
205 206 )|#
206 207 (defun calcularRestoFin (restoFin)
```

En este commit se hicieron cambios a la función `calcularRestoFin`, es un error aritmético debido a que el minuendo (la cual era el valor máximo que podía tomar los distintos colores) era siempre mayor al cálculo de restos, ocasionando que el resultado sea negativo. Como se puede observar en la imagen, en la franja verde, se hizo la debida corrección, cambiando el minuendo como el `restoFin` de la operación y haciendo corrección sobre los valores del sustraendo, tomando los mínimos de cada color (Ej: Antes si el rojo iba de 0 a 89, utilizaba el de 89 directamente como minuendo, ahora con la debida modificación no hace falta modificaciones en ese intervalo, pero si en el resto).

## Commit: 771ccd4

```
+
+
@@ -252,17 +253,19 @@ IMPACTO: No destructiva
252 253 )
253 254 +-----|#
254 255 (defun distribucionTemp (unix)
255 + (if (zerop (mod unix 225)) "[ 40,0% rojo| 1,3% rojo-intermitente | 53,3% verde| 1,3% verde-intermitente| 2,6% amarillo| 1,3% amarillo intermitente]"
256 + (if (zerop (mod unix 216)) "[ 40,28% rojo| 1,39% rojo-intermitente | 54,17% verde| 1,39% verde-intermitente| 1,39% amarillo| 1,39% amarillo intermitente]"
256 257 (format t "~| -%| -% rojo| -% rojo-intermitente| -% verde | -% verde-intermitente| -% amarillo| -% amarillo-intermitente|")
257 - (car (calcularPorcentajes (calcularRestoIni (mod unix 225)) (calcularRestoFin (mod (- 3600 (- 225 (mod unix 225))) 225))))
258 - (cadr (calcularPorcentajes (calcularRestoIni (mod unix 225)) (calcularRestoFin (mod (- 3600 (- 225 (mod unix 225))) 225))))
259 - (caddr (calcularPorcentajes (calcularRestoIni (mod unix 225)) (calcularRestoFin (mod (- 3600 (- 225 (mod unix 225))) 225))))
260 - (cadddr (calcularPorcentajes (calcularRestoIni (mod unix 225)) (calcularRestoFin (mod (- 3600 (- 225 (mod unix 225))) 225))))
261 - (caddddr (cde (calcularPorcentajes (calcularRestoIni (mod unix 225)) (calcularRestoFin (mod (- 3600 (- 225 (mod unix 225))) 225))))
262 - (car (last (calcularPorcentajes (calcularRestoIni (mod unix 225)) (calcularRestoFin (mod (- 3600 (- 225 (mod unix 225))) 225))))))
258 + (car (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
259 + (cadr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
260 + (caddr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
261 + (cadddr (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
262 + (caddddr (cde (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))
263 + (car (last (calcularPorcentajes (calcularRestoIni (mod unix 216)) (calcularRestoFin (mod (- 3600 (- 216 (mod unix 216))) 216))))))
263 264 )
264 265 )
```

En este commit, hubo cambios en la función `distribuciontemp`, debido un error conceptual en la anterior función, ya que tomabas la intermitencia como un tiempo aparte o separado de los colores principales (Ej: Tomábamos a rojo y rojo intermitente como valor independiente y no incluidos, respecto a la intermitencia, dentro de sus colores principales lo cual es erróneo, ya que ambos forman parte de uno solo)

### Commit ab4339e

```
9 9 |#
10 10 |=====
11 - #|(defun transicion (color-actual cambiar-a)
12 -   (cond ((or (and (equal color-actual 'en-verde) (equal cambiar-a 'amarillo)) (and (equal color-actual 'en-rojo) (equal cambiar-a 'amarillo))) (list color-actual "cambiar-a-amarillo"))
13 -         ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'rojo)) (list color-actual "cambiar-a-rojo"))
14 -         ((and (equal color-actual 'en-rojo) (equal cambiar-a 'verde)) (list color-actual "cambiar-a-verde"))
15 -         (t (list color-actual 'accion-por-defecto)))
11 + ( defun transicion (color-actual cambiar-a)
12 +   (cond
13 +     ((and (eq color-actual 'en-rojo) (eq cambiar-a 'en-rojo-intermitente)) (list color-actual "CAMBIAR-A-ROJO-INTERMITENTE" ))
14 +     ((and (eq color-actual 'en-rojo-intermitente) (eq cambiar-a 'en-verde)) (list color-actual "CAMBIAR-A-VERDE" ))
15 +     ((and (eq color-actual 'en-verde) (eq cambiar-a 'en-verde-intermitente)) (list color-actual "CAMBIAR-A-Verde-INTERMITENTE" ))
16 +     ((and (eq color-actual 'en-verde-intermitente) (eq cambiar-a 'en-amarillo)) (list color-actual "CAMBIAR-A-AMARILLO" ))
17 +     ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'en-amarillo-intermitente)) (list color-actual "CAMBIAR-A-AMARILLO-INTERMITENTE" ))
18 +     ((and (eq color-actual 'en-amarillo-intermitente) (eq cambiar-a 'en-rojo)) (list color-actual "CAMBIAR-A-ROJO" ))
19 +     (t (list color-actual 'accion-por-defecto ))
20 +   )
16 21 )
```

En este commit, hubo error conceptual en la función `transición`, en que se malinterpretó el orden del semáforo de “Rojo-amarillo-verde” y se corrigió a “Rojo-verde-amarillo”.

### **Bibliografía utilizada:**

<https://docs.scala-lang.org>

<https://docs.scala-lang.org/scala3/book/introduction.html>

<https://docs.scala-lang.org/scala3/book/taste-vars-data-types.html>

<https://docs.scala-lang.org/scala3/book/methods-most.html>

<https://docs.scala-lang.org/scala3/book/control-structures.html>

<https://docs.scala-lang.org/tour/pattern-matching.html>

<https://docs.scala-lang.org/scala3/book/tuples.html>

<https://docs.scala-lang.org/overviews/collections-2.13/concrete-immutable-collection-classes.html>

<https://docs.scala-lang.org/tour/pattern-matching.html>

<https://scala-lang.org/files/archive/spec/3.7/06-expressions.html>

<https://docs.scala-lang.org/scala3/book/control-structures.html>

